

Sutram SRM — Product Documentation Set

Deep Learning Super-Resolution Mapping from Medium-Resolution Satellite Imagery. Sentinel-2 10 m → 2.5 m with per-pixel trust.

Team Sutram · Smart India Hackathon · Generated 24 August 2026 · Contains: PRD · SRS · System Architecture · UI/UX Specification · Development Plan

What this system is, in one paragraph

Sentinel-2 gives free, frequently-updated imagery of all of India at 10 m per pixel — good enough to see a field, not good enough to see its boundary. Sutram SRM uses generative deep learning to reconstruct that imagery at 2.5 m, and — critically — ships a per-pixel confidence map alongside it, so an analyst always knows which sharpened details are supported by the sensor data and which the model inferred. It is a super-resolution system that tells you when not to trust it.

1. Product Requirements (PRD) — problem, users, goals, MVP, stories, metrics

2. Software Requirements (SRS) — functional requirements, rules, validation, security

3. System Architecture — stack, components, data flow, deployment

4. UI/UX Specification — journeys, screens, states, design system

5. Development Plan — roadmap, milestones, dependencies, Definition of Done

1. Product Requirements Document

1.1 Problem statement

Medium-resolution satellite imagery (10–30 m) is the backbone of operational Earth observation in India: it is free, covers the whole country, and refreshes every five days. But at 10 m a village road is one pixel wide, a small building is sub-pixel, and a field boundary is a blur. Analysts in agriculture, urban planning and disaster response therefore face a hard choice:

Option	Cost	Consequence
Use free 10 m imagery	Free, national coverage, 5-day revisit	Cannot resolve the features the decision actually depends on
Buy commercial sub-metre imagery	Thousands of rupees per scene	Unaffordable at state or national scale; infrequent revisit

Generative super-resolution promises a third option — but introduces a new and serious problem. A model that sharpens imagery is also a model that can *invent* imagery. An analyst shown a crisp building that the sensor never observed may make a real decision on a fabricated pixel. **This risk, not image quality, is the reason super-resolution is not already operational in most agencies.**

The problem we are actually solving

Not “make satellite images sharper” — several open models already do that. The unsolved problem is **making super-resolved imagery trustworthy enough to act on**: producing enhanced imagery together with a defensible, validated statement of which pixels are supported by observation and which are model inference.

1.2 Target users

User	Context	Needs from the system
Remote-sensing analyst (primary)	State agriculture / urban development dept, ISRO centre, research lab	Enhanced GeoTIFFs that load in QGIS/ArcGIS with correct georeferencing; explicit uncertainty; metrics they can defend in a report
Domain officer (secondary)	District agriculture officer, disaster response coordinator	Visual before/after for a specific area; a clear indication of where the imagery is reliable; no GIS expertise required
ML researcher (tertiary)	Academic or agency R&D	Reproducible benchmarks, swappable model branches, published validation protocol

1.3 Goals and non-goals

Goals	Non-goals
Produce ×4 super-resolved Sentinel-2 imagery (10 m → 2.5 m)	Compete with commercial sub-metre imagery on absolute resolution
Preserve geospatial footprint and spectral consistency exactly	Provide a general-purpose GIS platform
Quantify per-pixel uncertainty and validate that it predicts real error	Fully automated decision-making — a human stays in the loop
Run on free/low-cost compute so any Indian institution can adopt it	Real-time or on-orbit processing
Demonstrate value on Indian terrain specifically	Multi-sensor support (Landsat, SAR) in v1

1.4 Core features

#	Feature	Description	Priority
F1	Super-resolution engine	Four interchangeable model branches behind one interface: bicubic baseline, SEN2SR (fidelity), LDSR-S2 (generative detail), and our SEN2VENuS-trained model	P0
F2	Trust layer	Fuses LR-consistency, spectral angle, branch disagreement and sampling variance into a per-pixel confidence map in [0,1]	P0
F3	Geospatial integrity	CRS and footprint preserved to zero drift; output is a valid COG with overviews	P0
F4	Preprocessing	SCL cloud masking, reflectance scaling, overlap tiling with Hann feathering	P0
F5	Validation harness	Wald protocol, PSNR/SSIM/SAM/ERGAS, and a calibration curve proving confidence predicts error	P0
F6	Visual demo application	Browser UI: before/after comparison, confidence overlay, NDVI, metrics	P1
F7	Model training pipeline	SEN2VENuS ingestion, shard packing, disconnect-tolerant training	P1
F8	Downstream task demos	NDVI field boundaries, building edges, two-date change detection	P1
F9	AOI selection in-app	Draw a bounding box, fetch Sentinel-2, process end to end	P2
F10	Batch / API service	Queue many scenes; REST endpoint for programmatic access	P2

1.5 MVP scope

The MVP is deliberately narrow: **F1–F6 only**, RGB+NIR bands, on pre-downloaded scenes.

In MVP	Deferred, and why
4 bands (B04, B03, B02, B08) at ×4	20 m red-edge/SWIR bands — doubles data engineering for marginal demo value
Manually downloaded Sentinel-2 scenes	In-app AOI fetch — adds an external API dependency that can fail during a live demo
Single-user local application	Accounts, roles, multi-tenancy — no second user exists yet
Precomputed products in the demo	Live inference in-app — LDSR takes ~100 s/tile; unacceptable interactively
Streamlit UI	Custom React frontend — would consume the whole build budget for no added capability

Explicit anti-scope

No user accounts, no database, no cloud deployment, no mobile app, no real-time processing, no multi-sensor support in the MVP. Each of these is a plausible future feature and none of them help the system produce a trustworthy super-resolved pixel, which is the entire point.

1.6 User stories

ID	As a...	I want to...	So that...
US-1	Analyst	Upload a Sentinel-2 scene and get a 2.5 m GeoTIFF	I can see features my 10 m imagery cannot resolve
US-2	Analyst	Open the output in QGIS and have it align exactly with the input	I can overlay it on my existing map layers without reprojection
US-3	Analyst	See which pixels the model is unsure about	I never base a decision on invented detail
US-4	Analyst	Read a metrics report for the product	I can defend the output in a technical review
US-5	Officer	Compare before and after with a slider	I can immediately judge whether the enhancement helps my area
US-6	Officer	Toggle a red overlay of low-confidence regions	I can tell at a glance where to be cautious
US-7	Officer	View NDVI before and after	I can see sharper field boundaries without the vegetation signal shifting
US-8	Researcher	Benchmark all branches on the same tile	I can choose the right model for my use case
US-9	Researcher	Train the model on my own paired data	I can specialise it to my region
US-10	Researcher	Resume training after a disconnect	I can use free-tier compute without losing work

1.7 Success metrics

Metric	Target	Status
PSNR improvement over bicubic (Wald protocol)	$\geq +0.5$ dB	Met — SEN2SR +0.53 dB
Spectral fidelity (SAM)	$< 2.0^\circ$	Met — 1.904°
Geolocation drift	0 m	Met — 0.00 m measured
Confidence–error correlation	Negative and monotonic	Met — $r = -0.38$, error falls 4×
Own model trained on Indian terrain	Converged, benchmarked	Met — 5,400 KUDALIAR patches, val PSNR 39.23 dB
Inference speed (fidelity branch)	< 1 s per 128×128 tile	Met — 40 ms
Demo runs offline without failure	100% of dry runs	Met
Downstream task improvement	Measurable gain in boundary delineation	Pending — planned evaluation

1.8 Assumptions

- Sentinel-2 L2A remains freely available through the Copernicus Data Space.
- The ×4 degradation operator is approximately scale-invariant, so a model trained at 20 m → 5 m transfers to 10 m → 2.5 m.
- SEN2VENuS same-day pairs are co-registered well enough to train on without further alignment.
- Users have a GIS tool (QGIS is free) for consuming GeoTIFF output.
- A human reviews outputs before any operational decision.

1.9 Risks

Risk	Impact	Mitigation
Model hallucinates plausible but false structures	High	Trust layer with four independent signals; validated calibration; confidence band shipped inside the product file
Reviewer challenges the $\times 4$ claim given $\times 2$ training data	High	Documented openly: real HR target, PSF-based synthetic input, Wald protocol applied to training
Diffusion branch too slow for interactive use	Medium	Precompute demo products; fidelity branch runs in 40 ms for interactive paths
Free-tier compute interrupts training	Medium	Per-epoch checkpointing with full optimiser-state resume
Model underperforms on terrain unlike training data	Medium	Multi-site training; confidence map degrades gracefully and flags the shortfall
Upstream package churn (stale checkpoint URLs)	Low	Pinned fetch scripts; already encountered and handled once

1.10 Acceptance criteria

1. Given a valid 4-band Sentinel-2 GeoTIFF, the system outputs a 6-band COG at exactly $\frac{1}{4}$ the pixel size with identical bounds and CRS.
2. Bands 5 and 6 of every output contain sigma and confidence respectively.
3. Every branch is benchmarked on the same tile with PSNR, SSIM, SAM and ERGAS reported.
4. The calibration curve shows mean error decreasing as confidence increases.
5. The demo application runs with no network connection and displays imagery, confidence, NDVI and metrics.
6. Training can be interrupted and resumed with no loss beyond the current epoch.
7. All documentation regenerates from source.

2. Software Requirements Specification

2.1 Scope and interpretation

A note on shape

This is a scientific data-processing system, not a transactional web application. Requirement areas that a standard SRS template emphasises — accounts, roles, permissions, session management — are **deliberately minimal in the MVP**, because the MVP has exactly one user class operating a local tool. They are specified here for the deployed service variant so the design does not have to be retrofitted later, and each is marked with the release it applies to.

2.2 User roles and permissions

Role	Release	Capabilities	Restrictions
Local Operator	MVP	Full: run inference, train, benchmark, view demo, read/write local files	None — trusted single user on their own machine
Viewer	v2 (service)	View products and confidence maps; download COGs; read metrics	Cannot submit jobs or modify models
Analyst	v2 (service)	Viewer plus: submit scenes, choose branches, export reports	Cannot retrain or alter model registry
Administrator	v2 (service)	Analyst plus: manage model registry, quotas, purge products	Cannot alter audit logs

2.3 Functional requirements

Ingestion and preprocessing

ID	Requirement	Acceptance test
FR-1	Accept a GeoTIFF with exactly 4 bands in order B04, B03, B02, B08	A 3-band or 5-band input is rejected with a message naming the expected order
FR-2	Convert integer L2A DN to 0–1 reflectance on request (-- reflectance)	Input with max DN 10000 yields max reflectance 1.0
FR-3	Clip reflectance to [0,1]; negative values from atmospheric correction are removed	No output pixel is negative
FR-4	Mask SCL classes 0, 1, 3, 8, 9, 10 when an SCL band is supplied	Cloud-flagged pixels are excluded from processing
FR-5	Tile inputs of arbitrary size into 128×128 with 32 px overlap	A 1000×700 non-square input produces correct 4000×2800 output
FR-6	Blend overlapping tiles with a Hann window	No visible seam along tile boundaries in the output

Super-resolution

ID	Requirement	Acceptance test
FR-7	Provide four selectable branches: bicubic, SEN2SR, LDSR-S2, Ours	<code>--branches</code> accepts any comma-separated subset
FR-8	All branches expose <code>predict(lr) -> Prediction</code>	A new branch is usable with no change to pipeline or evaluation code
FR-9	Upscale by exactly $\times 4$	128 $\times$ 128 input yields 512 $\times$ 512 output
FR-10	LDSR-S2 produces a per-pixel sigma from N stochastic samples	With <code>--n-samples 8</code> , sigma is non-zero and varies spatially
FR-11	Support CPU, Apple MPS and CUDA	Identical output shape on all three; MPS materially faster than CPU

Trust layer

ID	Requirement	Acceptance test
FR-12	Compute LR-consistency by PSF-downsampling the SR output and differencing against the input	Bicubic scores worse than the hard-constrained branch
FR-13	Compute per-pixel spectral angle between input and downsampled output	Output is in degrees, finite everywhere
FR-14	Compute branch disagreement as normalised difference	Zero when both branches agree; bounded in [0,1]
FR-15	Fuse available signals into confidence in [0,1]	No value outside range; missing signals reweight rather than fail
FR-16	Report Δ NDVI introduced by super-resolution	Mean $ \Delta$ NDVI below 0.01 for the fidelity branch

Output and validation

ID	Requirement	Acceptance test
FR-17	Write a 6-band COG: 4 SR bands, sigma, confidence	<code>gdalinfo</code> reports 6 bands with named descriptions and overviews
FR-18	Preserve CRS exactly and rescale the transform so bounds are unchanged	Bound drift = 0.0 m; pixel size exactly $\frac{1}{4}$ of input
FR-19	Verify footprint after every write and record the result	Metrics JSON contains a <code>footprint.ok</code> field
FR-20	Implement Wald's protocol with PSF-based degradation	Degradation is not bicubic; documented and configurable
FR-21	Report PSNR, SSIM, SAM, ERGAS, RMSE per branch	All five present in <code>benchmark.json</code>
FR-22	Produce a calibration curve binding confidence to observed error	Correlation reported; bins include pixel counts

Training

ID	Requirement	Acceptance test
FR-23	Download SEN2VENuS sites with resume support	An interrupted download continues from its byte offset
FR-24	Pack patches into float16 shards, streamed not buffered	Peak memory independent of site size
FR-25	Support ×2 (native) and ×4 (PSF-degraded input) pairing modes	Manifest records which mode produced the shards
FR-26	Filter patches on contrast, not zero-fraction	Dark-biome sites yield a non-zero usable count
FR-27	Checkpoint every epoch and resume model, optimiser, scaler and epoch	Kill mid-run; resumed loss continues the prior trajectory
FR-28	Optimise L1 + LR-consistency + SAM + gradient with configurable weights	Each term is logged separately per epoch

2.4 Business rules

ID	Rule	Rationale
BR-1	No super-resolved product may be written without a confidence band	Prevents downstream use of sharpened pixels stripped of their caveat
BR-2	Geospatial footprint must never change	A shifted product silently corrupts every overlay analysis built on it
BR-3	Reflectance is never rescaled for aesthetics	Contrast stretching is display-only; analysis consumes physical values
BR-4	Cloud-flagged pixels are masked before, not after, super-resolution	Super-resolving cloud produces convincing, meaningless texture
BR-5	Generative-branch output is labelled visual-grade, not analysis-grade	Measured: it is the least LR-consistent branch
BR-6	Every published metric states its protocol and reference	PSNR without a stated degradation is unfalsifiable

2.5 Data requirements

Entity	Format	Key fields	Volume
Input scene	GeoTIFF	4 bands, CRS, affine transform, 10 m	~50 MB per 2000×2000 crop
Output product	COG	6 bands, 2.5 m, overviews	16× input pixels; ~5 MB per 512×512
Metrics record	JSON	per-branch timings, consistency, trust summary, footprint check	< 10 KB
Training shard	NPZ (float16)	<code>lr</code> , <code>hr</code> arrays	~400 MB per 1000 patches
Checkpoint	PyTorch <code>.pt</code>	model, optimiser, scheduler, scaler, epoch, history	~7 MB
Manifest	JSON	scale, per-site counts, shard names, split sizes	< 5 KB

2.6 Validation rules

Input	Validation	On failure
Band count	Exactly 4	Reject; state expected order B04/B03/B02/B08
CRS	Must be present and projected	Reject; a geographic CRS makes metre-based PSF meaningless
Reflectance range	Within [0,1] after scaling	Clip and warn
Raster size	≥ 1 pixel; padded up to one tile if smaller	Reflect-pad
Branch name	In {bicubic, sen2sr, ldsr, ours}	Reject; list valid names
Checkpoint path	Exists when <code>ours</code> selected	Clear message: train first or drop the branch
Device	In {cpu, mps, cuda} and available	Fall back to CPU with a warning
Training patch	Finite, contrast $\geq$ threshold, not saturated	Skip; count in manifest

2.7 Authentication and authorization

Release	Mechanism	Notes
MVP	None — OS file permissions only	Single-user local tool. Adding auth would be security theatre with no threat model.
v2 service	OAuth2 / OIDC against the hosting institution's identity provider	No bespoke password storage; institutions already have SSO
v2 service	Role claims mapped to Viewer / Analyst / Administrator	Authorisation checked server-side per endpoint, never in the client
v2 API	Scoped API tokens, rotatable, per-token rate limits	For programmatic batch access

2.8 Error handling and edge cases

Case	Behaviour
Input smaller than one tile	Reflect-pad to 128×128, process, crop back
Non-square raster	Handled by custom tiling (the upstream library's helper is known to size output from input width and corrupt this case)
Fully cloudy scene	Process, but report cloud fraction and flag the product
All-zero or flat patch	Confidence approaches its floor; no crash
Model checkpoint missing	Named error identifying which branch and how to fix
Out of memory on a large scene	Tiling bounds memory; document a recommended crop size
Interrupted download	Resume from byte offset on re-run
Interrupted training	Resume from <code>last.pt</code> ; at most one epoch lost
Upstream checkpoint URL changes	Fetch fails loudly with the attempted URL, not silently
Corrupt patch inside an archive	Skip, increment skip counter, continue

2.9 Security requirements

- **No credentials in the repository.** Copernicus access is interactive; no API keys are stored.
- **Model provenance is pinned.** Weights are fetched from named repositories at explicit paths and verified by size; a silently substituted model is a supply-chain risk for a system whose output informs decisions.
- **Checkpoints are loaded from trusted paths only.** PyTorch deserialisation executes code; loading an untrusted `.pt` is remote code execution.
- **Uploaded rasters are treated as untrusted input** — validated for band count, CRS and size before any processing.
- **v2 service:** TLS in transit, encryption at rest, per-user product isolation, audit log of every job submission and download.

2.10 Performance requirements

Operation	Target	Measured
Fidelity branch, 128×128 tile	< 1 s	40 ms (CPU)
Our model, 128×128 tile	< 1 s	20 ms (CPU)
Diffusion branch, 8 samples	Batch use only	~100 s (CPU)
Training epoch, 5400 patches	< 2 min	33 s (MPS)
Full 40-epoch run	< 1 h	22 min (MPS)
Dataset build, 6000 patches	< 10 min	~4 min
Demo page load	< 3 s	Sub-second (cached)
Peak memory, dataset build	Independent of site size	Bounded by shard size

2.11 Acceptance criteria

1. All FR-1 to FR-28 pass their stated acceptance tests.
2. Business rules BR-1 to BR-6 hold for every product the system emits.
3. Every validation rule produces a named, actionable error rather than a stack trace.
4. Every edge case in 2.8 is exercised at least once and does not crash the pipeline.
5. Performance targets in 2.10 are met on the reference hardware.

3. System Architecture Document

3.1 Architectural principles

Deliberately not a microservice platform

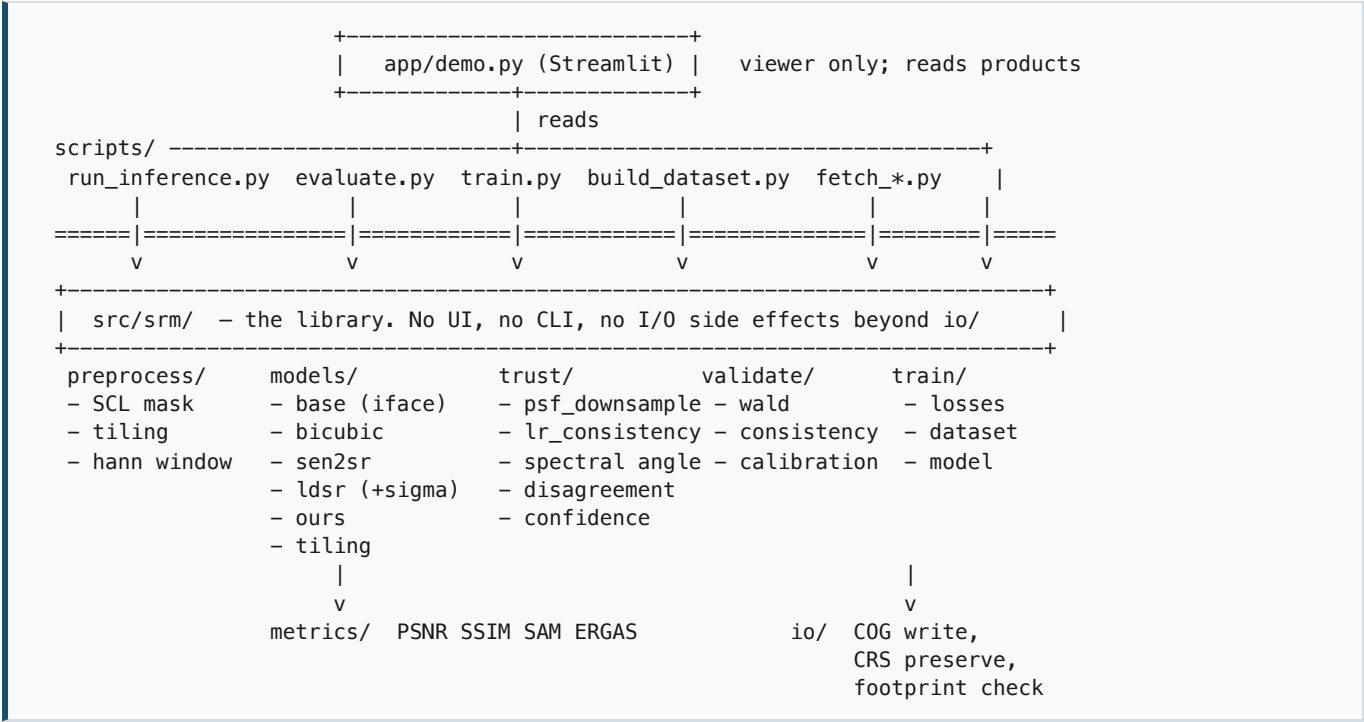
The MVP is a Python library with a thin UI. There is no database, no message queue, no container orchestration and no API gateway, because there is no requirement that needs them: one user, local files, batch processing. Every component below earns its place by serving a stated requirement. The v2 service design is included so the boundaries are right from the start — but it is explicitly *not built yet*.

- **Files are the interface.** GeoTIFF in, COG out. Any GIS tool, any language, no client library required.
- **One interface for all models.** Adding a branch touches no other component.
- **Uncertainty travels with the data**, inside the same file, not in a sidecar that can be lost.
- **Everything regenerates from source** — products, metrics and documentation.

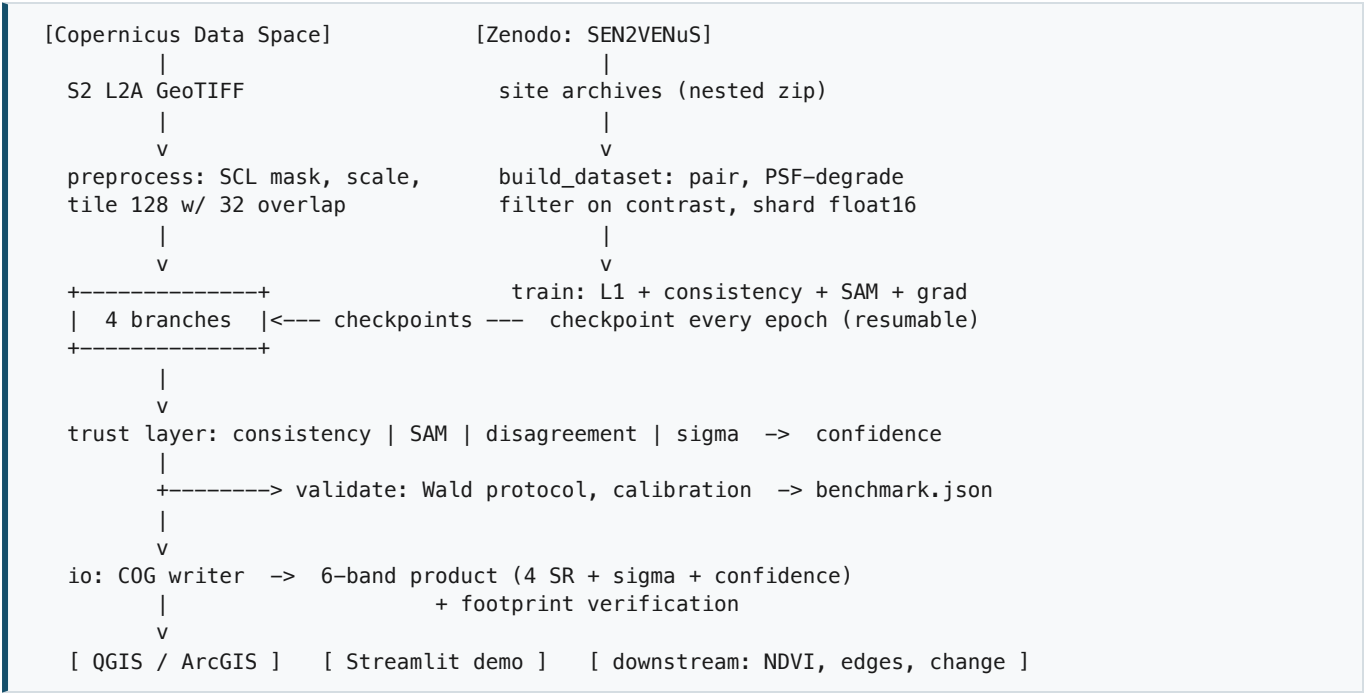
3.2 Technology stack

Layer	Choice	Why this and not the obvious alternative
Language	Python 3.12	The entire geospatial and ML ecosystem is here; 3.12 is required by <code>opens r-model</code>
ML framework	PyTorch 2.13	Both pretrained model families ship PyTorch weights; MPS support makes local training viable
Geospatial I/O	rasterio + rioarray (GDAL)	Correct CRS/affine handling and native COG writing; hand-rolling GeoTIFF is a footgun
Fidelity model	SEN2SR (SPAN CNN, 472k params)	Hard radiometric constraint built in; CC0; peer-reviewed (RSE 2025)
Generative model	LDSR-S2 latent diffusion (169M)	Native per-pixel uncertainty via sampling — the PS requirement, off the shelf
Our model	SPAN CNN fine-tuned	Warm start converges in minutes on free compute; a from-scratch transformer would not converge in the schedule
Frontend	Streamlit	A React SPA would consume the whole build budget and add no capability; the UI is a viewer, not an application
Packaging	uv + pyproject	Resolves and installs an order of magnitude faster than pip on this dependency tree
Docs	HTML → headless Chrome → PDF	Regenerates from source; no LaTeX toolchain to install

3.3 Component diagram



3.4 Data flow



3.5 Component responsibilities

Component	Responsibility	Does not
preprocess/	Masking, scaling, tile geometry, blend weights	Know about models
models/	Wrap each model behind <code>predict()</code> ; tiled inference	Write files or compute metrics
trust/	All uncertainty signals and their fusion	Depend on any specific model
validate/	Wald protocol, consistency check, calibration	Depend on the UI
metrics/	Reflectance-aware PSNR/SSIM/SAM/ERGAS	Assume 8-bit ranges
io/	COG write, CRS/transform preservation, footprint verification	Contain science logic
train/	Losses, shard dataset, model construction	Run inference pipelines
pipeline.py	Orchestrate branches → trust → product	Implement any of them

3.6 Storage

Path	Contents	Lifecycle
data/raw/	Input scenes, SEN2VENuS archives	Git-ignored; large
data/interim/	Packed training shards	Regenerable
data/outputs/	Products and metrics JSON	Regenerable
models/	Downloaded pretrained weights (~2.1 GB)	Git-ignored; re-fetchable
checkpoints/	Our trained model + history	The one artefact worth backing up
docs/	HTML sources and rendered PDFs	Version controlled

No database in the MVP. Product metadata lives in per-product JSON beside the raster, which keeps a product self-describing and portable — the file *is* the record.

3.7 Security architecture

Concern	MVP control	v2 service control
Input trust	Band/CRS/size validation before processing	Same, plus size quotas
Model provenance	Pinned URLs, explicit paths, size check	Signed model registry with hashes
Deserialisation	Load checkpoints only from local trusted paths	Registry-signed weights only
Secrets	None stored	Vault/KMS; no secrets in images
Transport	Local only	TLS 1.3
Isolation	OS user	Per-tenant object-storage prefixes
Auditability	Metrics JSON records provenance per product	Append-only job/download audit log

3.8 Deployment

Target	How	When
Local workstation	<code>uv venv + uv pip install -e . ; MPS/CPU</code>	MVP — this is the delivered form
Free Colab / Kaggle	Notebook clones the repo, checkpoints to Drive	MVP — training on borrowed GPUs
Institutional GPU server	Same package, <code>--device cuda</code>	v1.1 — larger training runs
Containerised batch service	Docker image, object storage, job queue	v2 — only when a second user exists

3.9 Monitoring and observability

Signal	Where	Why it matters
Per-branch runtime	Metrics JSON	Detects a branch silently falling back to CPU
LR-consistency per product	Metrics JSON	Needs no ground truth — the one quality signal available in production
Mean confidence, low-confidence fraction	Metrics JSON	A sudden drop indicates out-of-distribution input
Footprint check	Metrics JSON	Catches georeferencing regressions immediately
Training loss components	<code>history.json</code>	Separately logged so a diverging term is visible, not hidden in a total
Skipped-patch count	Manifest	A spike means a bad filter or corrupt archive

The production quality signal

LR-consistency is the architecturally important metric: it compares the product against its own input, so it runs on every real scene where no ground truth exists. It is the difference between a system that was validated once and a system that reports its own health continuously.

3.10 Scalability

Dimension	Current limit	Path forward
Scene size	Memory-bounded by tiling; ~2000×2000 comfortable	Windowed read/write via rasterio for full 10980 ² tiles
Throughput	One scene at a time	Tiles are independent — embarrassingly parallel across processes or nodes
Model size	472k params, CPU-viable	Larger backbones only if benchmarks justify the cost
Training data	6000 patches, 2.3 GB	Streaming shards already support arbitrary site counts
Users	One	v2 service with a job queue — deferred until a second user exists

4. UI/UX Specification

4.1 Design principles

Principle	What it means here
Uncertainty is not an advanced feature	Confidence is surfaced on the main screen, not buried in a settings panel. The whole product claim depends on it being seen.
The comparison is the interface	A super-resolved image alone means nothing. Every view shows before and after together.
Never let a demo depend on the network	The app reads precomputed products. No model call, no download, no API during a presentation.
Honest visuals	Contrast stretching is labelled display-only. The UI never makes output look better than the data supports.
No decoration	No splash screens, no animation, no gradients. Every pixel of chrome competes with the imagery the user came to see.

4.2 User journeys

Journey A — Officer evaluating an area (primary demo path)

```
open app -> product already selected -> IMAGERY tab
  | sees 10 m and 2.5 m side by side; reads the inference disclaimer
  v
TRUST tab -> confidence map + red overlay; drags threshold slider
  | understands which sharpened detail is supported
  v
NDVI tab -> field boundaries sharper, NDVI mean shift ~0
  | confirms the vegetation signal did not move
  v
decision: "this is usable for my district"      [~90 seconds, no GIS skill]
```

Journey B — Analyst producing a product

```
CLI: run_inference.py --input scene.tif --branches sen2sr,ldsr
    -> product.tif (6 bands) + metrics.json
    -> open in QGIS, overlay on existing layers, confirm alignment
    -> read metrics.json: consistency, footprint.ok, trust summary
    -> cite figures in report
```

Journey C — Researcher benchmarking

```
evaluate.py --branches bicubic,sen2sr,ours
    -> metric table + calibration curve printed and written to JSON
    -> METRICS tab in app renders the same numbers for presentation
```

4.3 Navigation and screens

Flat, four-tab structure. No nesting, no modals, no routing — the entire app is one screen with four views of the same product.

SIDEBAR	Deep Learning Super-Resolution Mapping Sentinel-2 10 m -> 2.5 m with per-pixel trust		
Product [v]	[Mean confidence]	[Low-conf area]	[Footprint]
CRS EPSG:32643	0.805	2.7%	yes
Pixel 2.5 m	(Imagery Trust layer NDVI Metrics)		
Bands 6	[tab content]		

Screen	Content	Primary action
Header	Title, subtitle, three KPI tiles	Orientation at a glance
Imagery	Input (10 m) and output (2.5 m) side by side; inference disclaimer	Visual comparison
Trust layer	Confidence map; SR image with sub-threshold pixels in red; threshold slider	Judge reliability
NDVI	NDVI before/after; mean-shift metric	Verify spectral integrity
Metrics	Wald table; calibration bar chart; per-branch consistency JSON	Defend the result

4.4 Key interactions

Interaction	Behaviour	Rationale
Product selector	Dropdown of every COG in <code>data/outputs/</code> ; reloads all tabs	Fast switching between demo areas
Confidence threshold slider	0.00–1.00, step 0.05; red overlay and the percentage caption update live	Lets a sceptical viewer set their own bar rather than trusting ours
Tab switching	Instant; state cached	No spinner between views
Image display	2–98 percentile stretch, labelled display-only	Dark scenes are unreadable unstretched; the label prevents misreading

4.5 States

State	Trigger	What the user sees
Empty	No products in <code>data/outputs/</code>	“No products found. Run <code>python scripts/run_inference.py</code> first.” — names the exact command
Loading	First read of a large COG	Spinner; results cached thereafter
Partial	Product has no confidence band	Trust tab shows a warning instead of failing; other tabs work
Partial	No benchmark JSON present	Metrics tab hides the table, keeps consistency
Error	Corrupt or unreadable raster	Named error identifying the file
Nominal	Everything present	All four tabs populated

4.6 Visual design system

Token	Value	Use
Primary	#16506b deep teal-blue	Headers, table headers, accents
Alert	#ff0000 pure red	Low-confidence overlay only — reserved, never decorative
Success	#1d7a3e	Passing metrics
Warning	#96650d on #fdf6ec	Caveats and scope warnings
Surface	#f7f9fa / #f4f8fa	Code blocks, callouts
Body text	#1a1a1a	Maximum legibility on projectors
Type	System sans; monospace for paths and numbers	Tabular figures so metric columns align
Spacing	4 px base scale	Consistent rhythm

4.7 Responsive behaviour

Viewport	Layout
≥ 1200 px (demo projector)	Two-column image comparison, sidebar expanded — the target
768–1199 px (laptop)	Columns narrow; images scale to container width
< 768 px (tablet)	Columns stack vertically; sidebar collapses to a drawer
Wide tables	Horizontal scroll within their own container; the page never scrolls sideways

4.8 Accessibility

- **Never colour alone.** Low-confidence regions carry a numeric percentage caption as well as the red overlay — essential given red/green colour-vision deficiency affects roughly 8% of men.
- Body text contrast ≥ 4.5:1; the primary colour on white measures ~8.6:1.
- All controls keyboard reachable; the threshold slider is arrow-key adjustable.
- Every image carries a descriptive caption stating resolution and content.
- Metric tables are real tables with headers, not images — screen-reader navigable.
- Confidence rendered as greyscale intensity, legible without colour perception.

5. Development Plan

5.1 Roadmap overview

M0	Foundation	[DONE]	repo, env, geospatial I/O, metrics
M1	Pipeline MVP	[DONE]	4 branches, trust layer, COG product, demo
M2	Own model	[DONE]	SEN2VENuS ingest, training, benchmark
M3	Validation depth	[NEXT]	downstream tasks, India reference, ablations
M4	Hardening		full-tile inference, tests, packaging
M5	Service (v2)		API, queue, auth – only if a second user appears

5.2 Completed work

Milestone	Delivered	Evidence
M0	Python 3.12 env, package layout, COG writer with CRS preservation, reflectance-aware metrics	Footprint drift measured at 0.00 m
M1	Preprocessing, four-branch interface, Hann-feathered tiling, trust layer, Wald validation, Streamlit demo	Calibration $r = -0.38$; error falls 4× across confidence bins
M2	SEN2VENuS fetch/pack/train, custom loss, our benchmarked model	5,400 KUDALIAR patches; val PSNR 39.23 dB; best SSIM and SAM on the reference tile

5.3 Remaining work

M3 — Validation depth P0

Task	Depends on	Definition of Done
Downstream task evaluation: NDVI field-boundary and building-edge delineation on LR vs SR	M1	A quantified gain (IoU or boundary F1) reported for at least one task, or an honest null result
India reference validation against Bhoonidhi LISS-IV (5.8 m)	M2	≥ 2 overlapping AOIs compared; agreement reported with the acquisition gap stated
LDSR-S2 included in the full benchmark	M2	Four-branch table with sigma-based uncertainty in the comparison
Multi-site training run (add ANJI, MAD-AMBO, SUDOUÉ-4, ESTUAMAR)	M2	Benchmarked against the KUDALIAR-only model; generalisation gap quantified
Loss ablation (drop consistency / SAM / gradient in turn)	M2	Table showing each term's contribution — turns a design claim into evidence

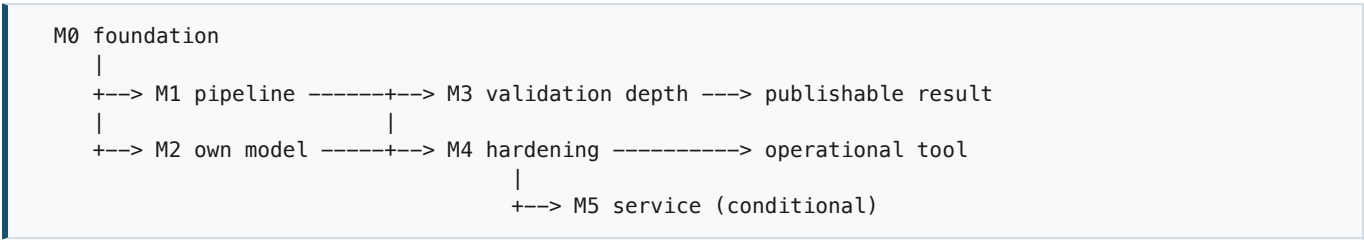
M4 — Hardening P1

Task	Depends on	Definition of Done
Windowed full-tile inference	M1	A full 10980 ² Sentinel-2 tile processes without exhausting memory
Automated test suite	M1	Tests cover footprint preservation, band-order validation, tiling on non-square input, resume correctness; all pass in CI
Copernicus fetch by AOI	M1	Bounding box + date range yields an analysis-ready 4-band stack
Cloud masking wired into the CLI end to end	M1	Scene with SCL produces a masked product and a reported cloud fraction
Reproducible environment lock	M0	A clean machine reproduces the benchmark numbers from the lockfile

M5 — Service P2, conditional

Task	Trigger	Definition of Done
REST API, job queue, object storage	A second concurrent user exists	Submit a scene, poll status, download product
OIDC auth and role mapping	Multi-user deployment	Viewer/Analyst/Administrator enforced server-side
Containerisation and deployment	Institutional hosting agreed	One-command deploy; health endpoint; audit log

5.4 Dependencies



Dependency	Type	Risk if it fails
Copernicus Data Space	External	No new input scenes; existing demo unaffected
Zenodo (SEN2VENuS)	External	Cannot retrain; already downloaded locally
HuggingFace model weights	External	Pretrained branches unavailable; already cached
Bhoonidhi / NRSC	External	India reference validation deferred — the only blocking dependency in M3
sen2sr, opensr-model	Library	Version churn; mitigated by pinning (a stale checkpoint URL has already been hit once)

5.5 Definition of Done

A task is done when all of these hold

#	Criterion
1	The code runs end to end from the project root on a clean environment
2	It is exercised on real data, not only synthetic — a passing test on constructed input proves nothing about a satellite scene
3	Any geospatial output passes <code>check_footprint()</code> with zero drift
4	Any quality claim is backed by a number produced by <code>evaluate.py</code> , with the protocol stated
5	Failure modes produce named, actionable errors — never a bare traceback
6	Non-obvious decisions are explained in a comment answering <i>why</i> , not restating <i>what</i>
7	README or QUICKSTART is updated if the user-facing surface changed
8	Committed with a message explaining the reasoning, not just the diff
9	Negative results are reported as prominently as positive ones

5.6 Priorities if time runs short

Rank	Work	Why this order
1	Downstream task evaluation	Converts “PSNR improved” into “the analysis got better” — the single most persuasive evidence available
2	Loss ablation	Cheap (hours), and turns the loss design from an assertion into a finding
3	LDSR in the full benchmark	Completes the four-branch story
4	Multi-site training	Tests the generalisation claim directly
5	India reference validation	Highest credibility value, but gated on external data access
6	Full-tile inference	Engineering, not science; defer if time is tight

5.7 Current status summary

Area	State	Evidence
Pipeline	Complete	Four branches, end to end, verified
Trust layer	Complete and calibrated	$r = -0.38$; error $0.0366 \rightarrow 0.0087$ across bins
Geospatial integrity	Verified	0.00 m drift on every product
Own model	Trained and benchmarked	Best SSIM (0.8346) and SAM (1.845°) of three branches
Demo	Working offline	Four tabs, precomputed products
Documentation	Generated from source	Pipeline doc + this set
Downstream evaluation	Not started	Highest-value remaining work
India reference validation	Not started	Gated on Bhoonidhi access
Test suite	Not started	Manual verification only so far

5.8 Headline results to date

Branch	PSNR	SSIM	SAM°	ERGAS	Time
Bicubic	32.68	0.7965	2.025	3.735	1 ms
SEN2SR	33.21	0.8194	1.904	3.500	40 ms
Ours	31.57	0.8346	1.845	4.069	20 ms

How to state this honestly

Our model achieves the **best structural (SSIM) and spectral (SAM) fidelity** of the three branches, trading raw PSNR for the properties that matter in remote sensing — which is exactly what its loss function optimises. It does *not* beat SEN2SR on PSNR or ERGAS, and we do not claim it does.

Two caveats to state before a reviewer raises them. First, this evaluation tile is European, while our model was trained only on Indian terrain (KUDALIAR) — it is being tested out-of-domain and still leads on two of four metrics. Second, PSNR is a per-pixel L2 measure and is well known to correlate weakly with interpretability; SSIM and SAM are the metrics a remote-sensing reviewer weighs. On its own KUDALIAR validation split the model reaches 39.23 dB PSNR and 0.9524 SSIM.